

Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science



Year 11 2026

Programming 3 — Answers

Data types, casting, the four traps and the Character Builder — OCR J277 §2.2.2

Ask Python what type something is — answers

You typed	It printed	Why
<code>type(7)</code>	<code>int</code>	A whole number.
<code>type(9.81)</code>	<code>float</code>	The decimal point makes it a Real.
<code>type("Batman")</code>	<code>str</code>	Quote marks mean text.
<code>type(True)</code>	<code>bool</code>	One of only two possible values.
<code>type("7")</code>	<code>str</code>	The quote marks win. It looks like a number to you; to Python it is text.
<code>type(input(...))</code>	<code>str</code>	Always. Whatever the user types — even 42 — arrives as text.

Those last two are the same fact twice, and it is the most useful thing on the sheet. **Quote marks, and `input()`, both give you text.**

Choosing the right type — answers

What you need to store	Data type	Why
Number of students in a class	Integer	You count them in whole people. Half a student does not exist.
Price of a train ticket	Real	£24.50 needs the pence. An Integer would lose them.
Whether a user is logged in	Boolean	There are exactly two possibilities and no others.
Exam grade, as one letter	Character	Exactly one character. (<i>In Python this is just a String of length 1 — but OCR names Character as its own type, so use that word in the exam.</i>)
UK mobile phone number	String	See the box below — this is the one people get wrong.

Why a phone number is text and not a number

Two reasons, and you need both for full marks.

1 — The leading zero disappears. Storing 07700 900123 as an Integer gives you 7700900123. The 0 is gone, because numbers do not have leading zeros. It is no longer a phone number.

2 — You never do arithmetic on it. You never add two phone numbers, or average them. **If you will never do sums with it, it is a String** — and the same rule covers postcodes, bank account numbers and student ID numbers.

The four traps — what actually happened

Trap 1 — input() always gives you a String. Even when the user types a number. That is the entire reason int() exists.

Trap 2 — int() cannot swallow a decimal.

```
print(int("3.5"))  
ValueError: invalid literal for int() with base 10: '3.5'
```

You might have expected 3. **Python refuses rather than guessing** — it will not silently throw away your .5. If you want decimals, use float().

Trap 3 — you cannot glue a number onto text with +.

```
strength = 8  
print("My strength is " + strength)  
TypeError: can only concatenate str (not "int") to str
```

The fix is str(strength) — cast the number into text first, then + works.

Trap 4 — the one that does not warn you.

```
print("3" * 4)  
3333
```

No error. *** on a String means “repeat it”**, so you get 3333 and not 12. The program ran perfectly and gave the wrong answer — a **logic error**, and the hardest kind to spot.

The bool() question — and the rule behind it

```
print(bool("False")) -> True  
print(bool(""))      -> False  
print(bool(0))        -> False
```

The print() is not decoration. The online editor runs your code as a program, so a line reading only bool(0) works the answer out and throws it away — you get an empty output box and no error.

bool("False") is True, and that surprises everybody.

Python is not reading the word. It is asking a much blunter question: **is there anything here?** "False" is a String with five characters in it, so there *is* something there — so it is True.

The rule: empty text and zero are False. **Everything else is True.** The word inside the quote marks is irrelevant.

Challenge 1 — My Superhero

```
name = "Batman"
location = "Gotham City"
strength = 8
speed = 6
agility = 7
height = 1.88

print("My name is " + name + " and I live in " + location + ".")
print("My strength is " + str(strength) + " out of 10.")
print("I am " + str(height) + " m tall.")
print("Power rating: " + str((strength + speed + agility) / 3))
```

Every number needed `str()` to go inside a sentence. `name` and `location` did not — they were already text.

The surprise in the average — and it is examinable

`strength`, `speed` and `agility` are all **Integers**. The average of 8, 6 and 7 is 7 — a whole number. But Python printed `7.0`, not 7.

Dividing always produces a Real, whatever you divide. Two whole numbers in, a Real out, every time — even when it goes exactly. `24 / 3` gives `8.0`, never 8.

So you did not type a Real anywhere in that line, and you got one anyway. **That is the data type arriving on its own.**

Challenge 2 — Weight on the Moon

```
weight = float(input("What do you weigh in kg? "))
moon = weight * 0.165
print("On the Moon you would weigh " + str(moon) + " kg")
```

The cast is `float()`, not `int()` — because people weigh things like 70.5 kg, and `int("70.5")` would crash exactly as it did in Trap 2. Entering 70.5 gives 11.6325.

Challenge 3 — The Character Builder

```
name      = input("Character name: ")
level     = int(input("Level: "))
health    = float(input("Health remaining: "))
rank      = input("Rank (one letter): ")
strength  = int(input("Strength out of 10: "))
speed     = int(input("Speed out of 10: "))

power = (strength + speed) / 2

print("--- CHARACTER SHEET ---")
print("Name:    " + name + " (rank " + rank + ")")
print("Level:   " + str(level))
print("Health:  " + str(health))
print("Power:   " + str(power))
```

Entering Ripley, 7, 87.5, A, 8, 6:

```
--- CHARACTER SHEET ---
Name:  Ripley  (rank A)
Level:  7
Health: 87.5
Power:  7.0
```

Line	Type	Cast	Why that one
name	String	<i>none</i>	Already text. Casting it would do nothing.
level	Integer	<code>int()</code>	Levels are whole numbers.
health	Real	<code>float()</code>	87.5 needs the decimal.
rank	Character	<i>none</i>	One character — still text.
strength	Integer	<code>int()</code>	Whole score out of 10.
power	Real	<i>none</i>	You never cast it. Dividing made it a Real.

Asking for a Level of 3.5

```
ValueError: invalid literal for int() with base 10: '3.5'
```

`int()` rejected it, and that is correct behaviour — you told Python a level is a whole number, so 3.5 is not a valid level.

Choosing a data type is a decision about what counts as valid. That is why §2.2.2 asks you to pick a suitable type for a scenario: the type is not decoration, it is a rule.

Note what it did *not* do. It did not round to 3, or to 4, or guess. Python would rather stop than quietly change your data.

The stretch — a real Boolean

```
answer = input("Still alive? (yes/no) ")
alive = (answer == "yes")

print(alive)           # True
print(type(alive))     # <class 'bool'>
```

`==` asks a question, and the answer to a question is a **Boolean**. So `(answer == "yes")` is not a comparison you *do* — it is a value you can *store*, and its type is `bool`.

Why not just keep the text "yes"? Because "no" is also a non-empty String, and you saw in Trap 4's box what `bool()` makes of that — `True`. Store the answer as text and every player is alive forever.